ADTT

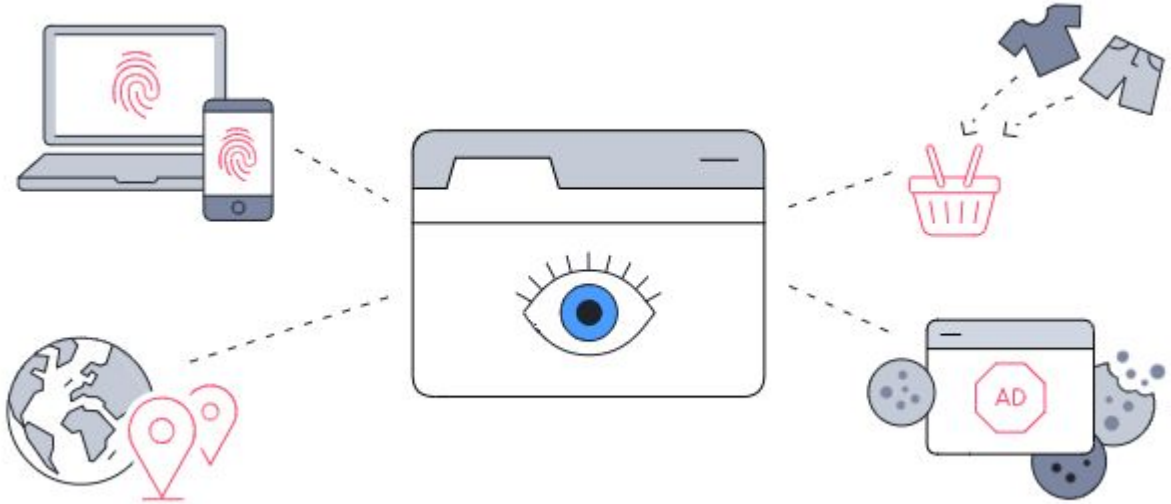
Identifying JavaScript Trackers via Interaction Simulation and Dynamic Taint Tracking

Jaeho Bae

Department of Computer Science and Engineering

Background: What is ATS?

- Advertising and tracking service (ATS) raises **privacy concerns**
  - ➢ Modern websites extensively embed ATS JavaScript files
  - ➢ Filterlist- or ML- based blocking techniques have been proposed



Background: Existing ATS Blocking

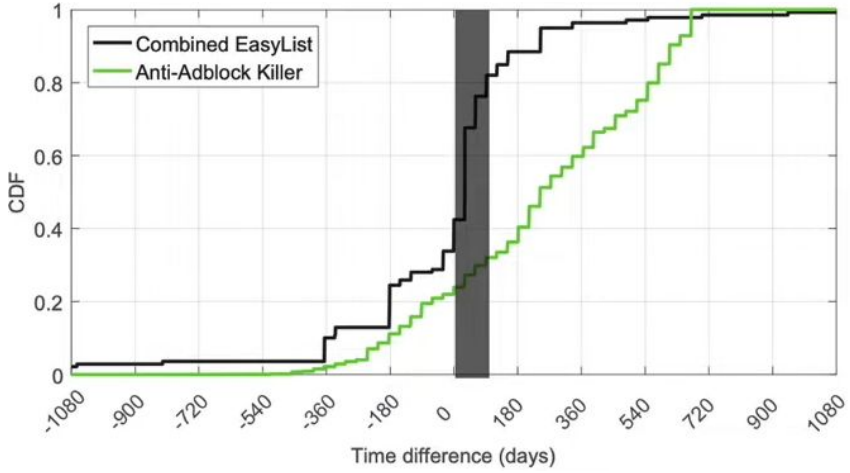
- **Filter Lists:**
  - ➢ e.g., Easy List, Easy Privacy, Adblock, uBlock Origin, etc.
  - ➢ Relies on **manually curated** URL filter lists
  - ➢ **Easily evaded** by changing the resource's URL

- **ML-based:**
  - ➢ e.g., AdGraph, WebGraph, AdCPG
  - ➢ Learns the **page context** (DOM, JS, network relationships)
  - ➢ Robust against URL-based evasion

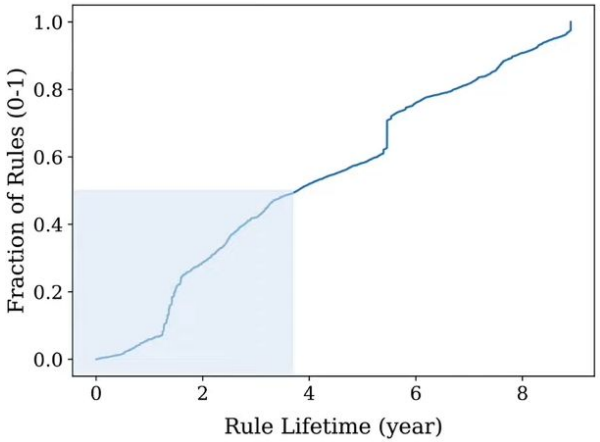
Background: Existing ATS Blocking

- **Filter Lists:**
  - ➢ e.g., Easy List, Easy Privacy, Adblock, uBlock Origin, etc.
  - ➢ Relies on **manually curated** URL filter lists
  - ➢ **Easily evaded** by changing the resource's URL

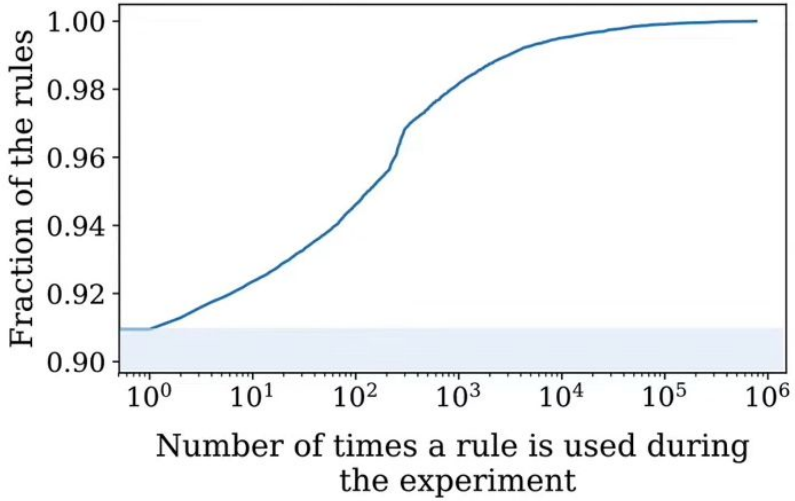
3 months to add new rules



3.8 years to remove rules



90% rules are useless



Background: Existing ATS Blocking

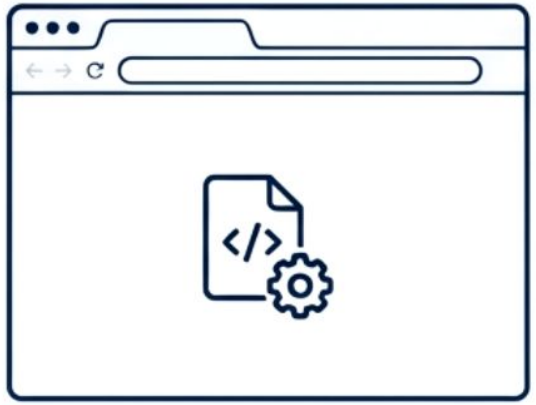
- **ML-based:**
  - ➢ Model the structure and execution behavior of web pages
  - ➢ Learns the **context of webpage**
  - ➢ Graph-based approaches
    - ■ Learns relationships among web elements
      (e.g., HTML contexts, storage access, script execution, network requests)
    - ■ e.g., AdGraph, WebGraph, AdFlush
  - ➢ JavaScript-based approaches
    - ■ Learns the syntax and semantics of individual ATS scripts
    - ■ e.g., AdCPG, NoT.js
  - ➢ ML-based research focuses almost exclusively on detecting ATS resources at the **Initial Page Load**

Background: Dynamic Taint Tracking (DTT)

- **Definition:**
  - ➢ A technique for monitoring the data flow of sensitive and untrusted information during runtime
- **How it works:**
  - ➢ **Taint Sources**: APIs that emit sensitive data
    - e.g., navigator.userAgent, URL.href
  - ➢ **Taint Sinks:** Sensitive APIs where data is leaked
    - e.g., navigator.sendBeacon, HTMLImageElement.src
  - ➢ It tracks **tainted data propagation** from source to sink to detect information leak

Introduction: Blind Spot of Prior Works

Initial Page Load



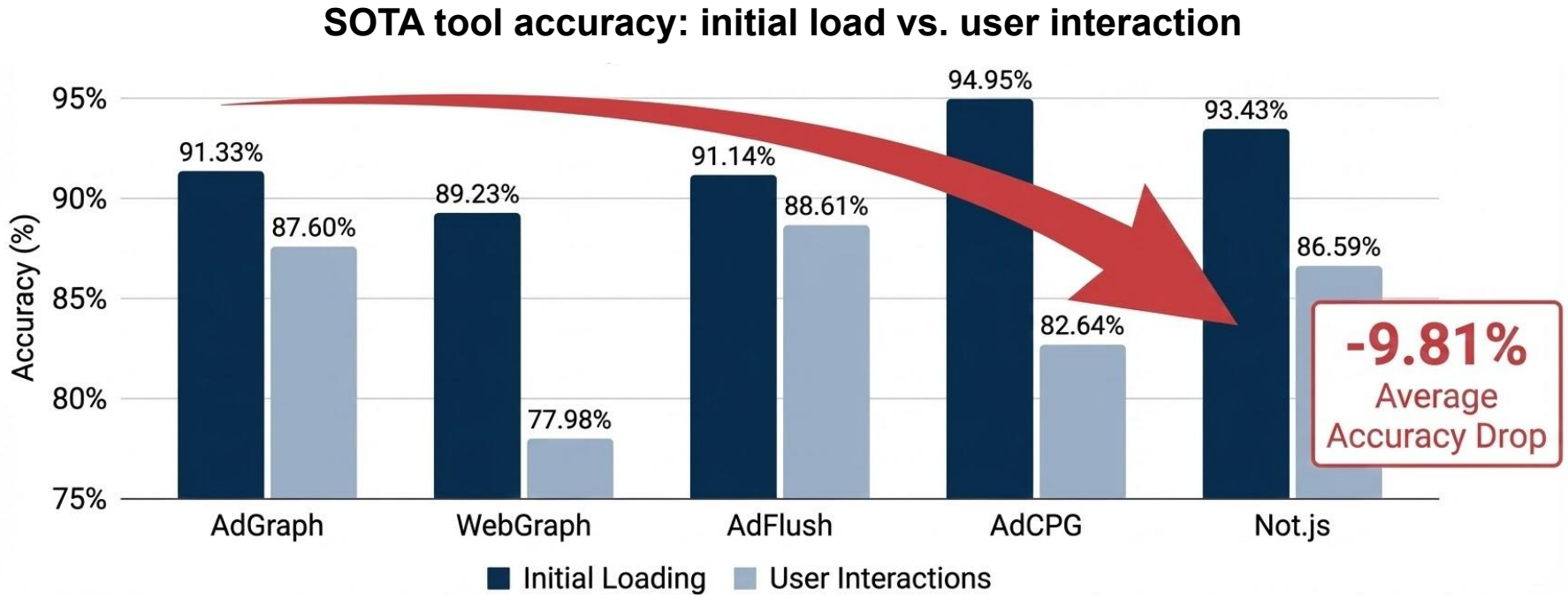
- Filterlist-based approaches are labor-intensive and easy to circumvent
- ML-based blockers focus on detecting ATS resources at the **Initial Page Load**

User Interaction

- But a massive, hidden threat emerges after user's interact (e.g., click, scroll, type)
- Our analysis shows these **interaction-triggered ATS scripts are 3.7x more prevalent** than those at initial load

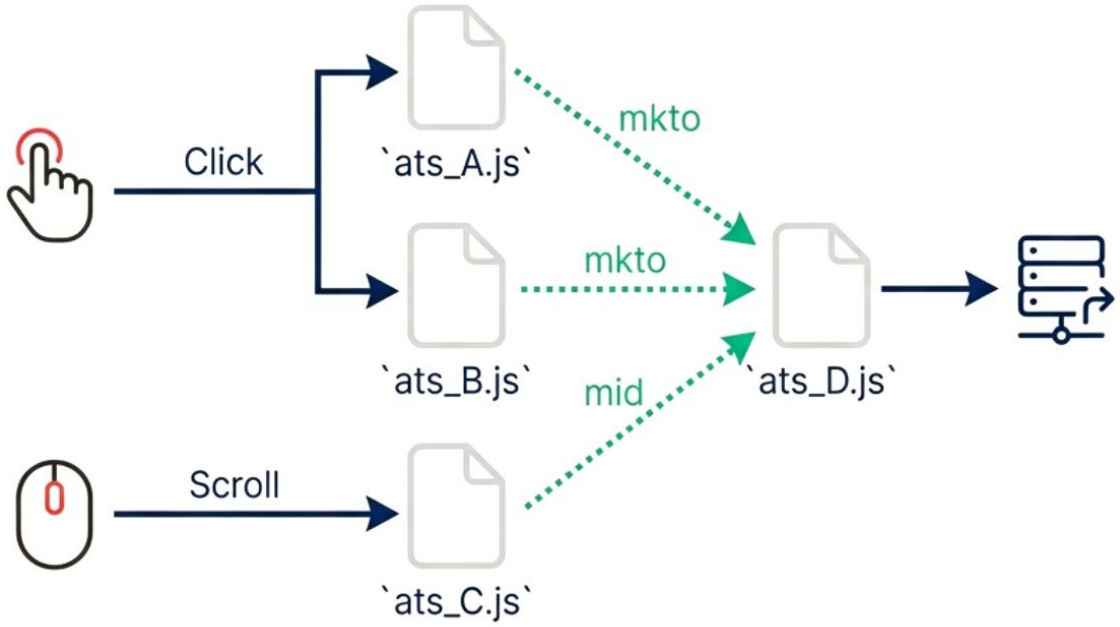
Introduction: Crumble Under Interaction

- We tested state-of-the-art (SOTA) ML-based blockers
- They perform well on initial page loads. But when faced with interaction-triggered scripts, their **accuracy plummets**



Introduction: Fragmented Scripts

Why They Fail:
The "Fragmented Script" Evasion Tactic



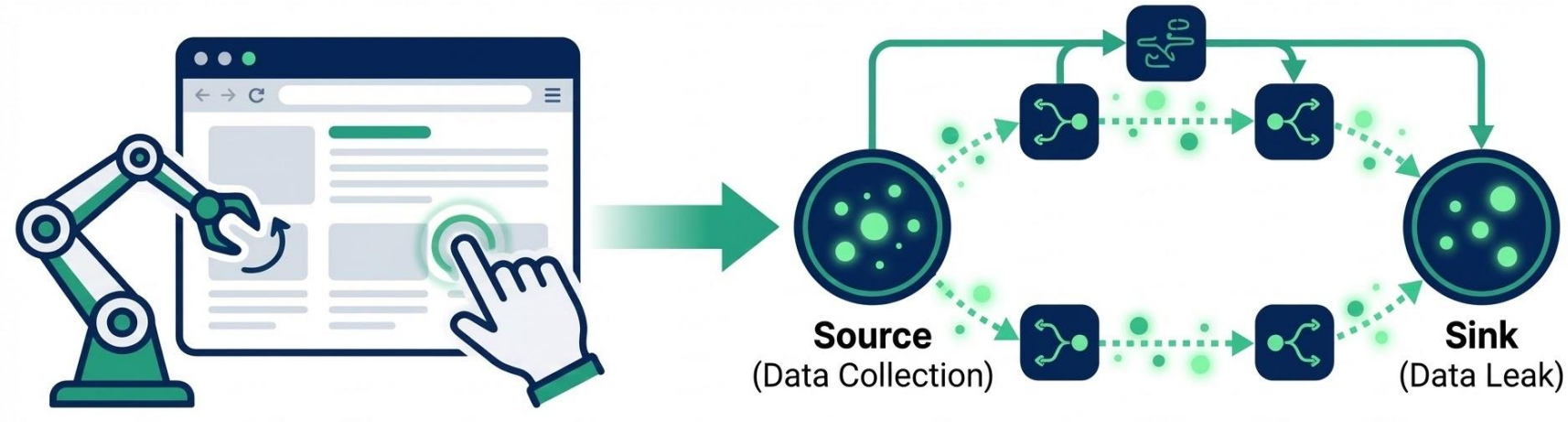
Examples of ATS scripts

- ATS scripts split logic across multiple, often from different origins
- These **fragments** activate dynamically after different user interactions
- They **cooperatively collect, process, and exfiltrate** user's private data
- **67.13%** of interaction-triggered trackers use this fragmented approach

Approach: ADTT

- ADTT **simulate interactions** with web pages as if they were real users, and capture the sensitive data flows using **dynamic taint tracking (DTT)** technology
- It allows us to detect the entire chain from source to sink, even if the script is fragmented or dynamically loaded

Combine interaction simulation and dynamic taint tracking (DTT)

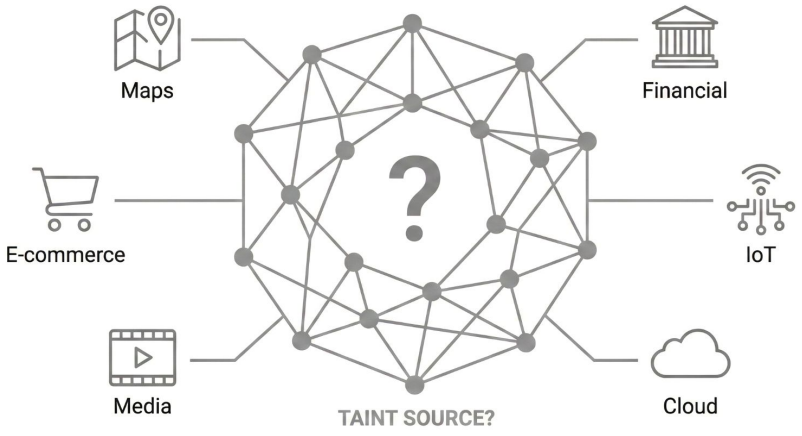


Approach: ADTT's Two Critical Challenge

- **Challenge 1: Interaction Selection**
  - ➢ There are **124 distinct types of user interactions** on the web
  - ➢ Exhaustively testing all possible combinations is infeasible
- **Challenge 2: Taint Source Identification**
  - ➢ ATS scripts use **a very wide range of APIs** to collect information
  - ➢ It is difficult to comprehensively and precisely define all relevant source APIs

User interactions types

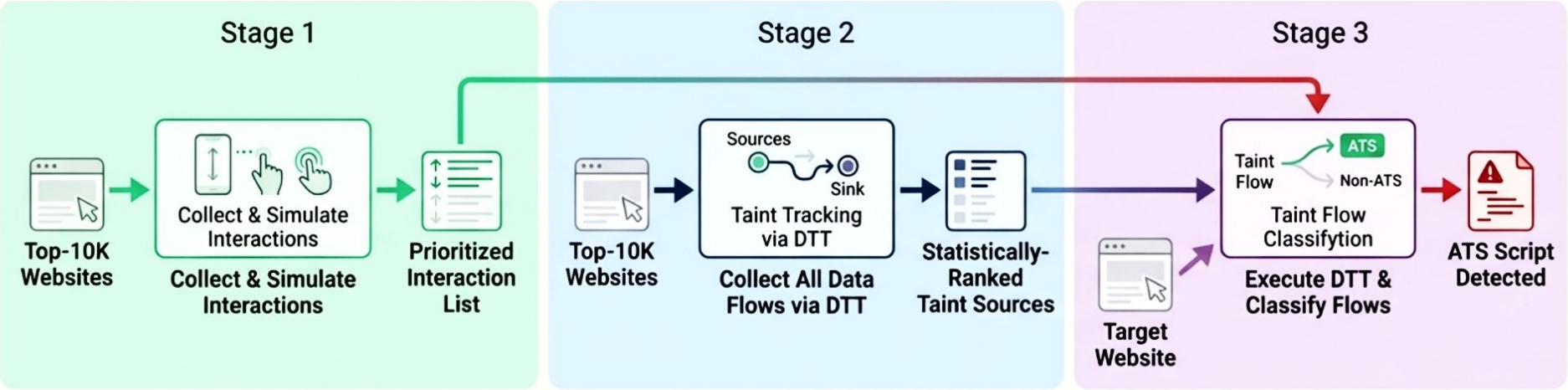
Thousands of APIs are potential taint sources



Design: ADTT Overall Workflow

- **ADTT consists of three core components:**
  - ➢ **Interaction Scheduler**: Prioritizes user interactions that are most likely to load JS resources, reducing the search space
  - ➢ **Source Identifier**: Statistically identifies taint sources (APIs) that are strongly associated with ATS behaviors
  - ➢ **ATS Detector**: Run DTT based on the identified interactions and sources/sinks to detect ATS scripts

ADTT workflow

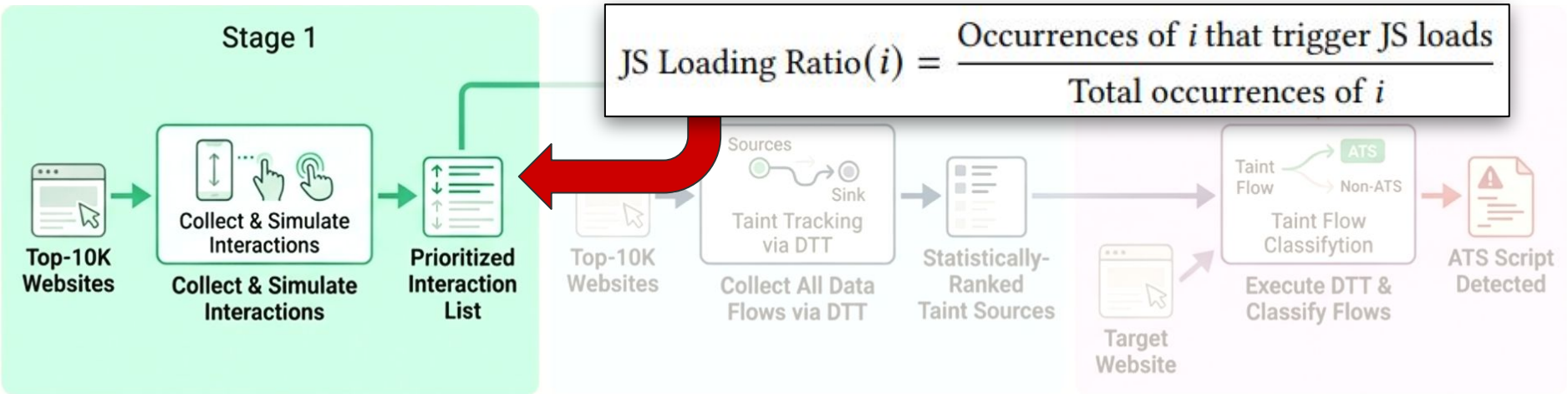


Design: (1) Interaction Scheduler

- **Challenge:** 124 different interaction types on the web, and testing every combination is impractical
- **Solution:** Reduce the interaction space by simulating interactions with **high JS loading ratios** to maximizes detection efficiency

⇒ We prioritize the **user interaction sequences** (e.g., keydown, click→click, dragstart) that significantly contribute to JS loading.

Stage 1. Interaction Scheduler



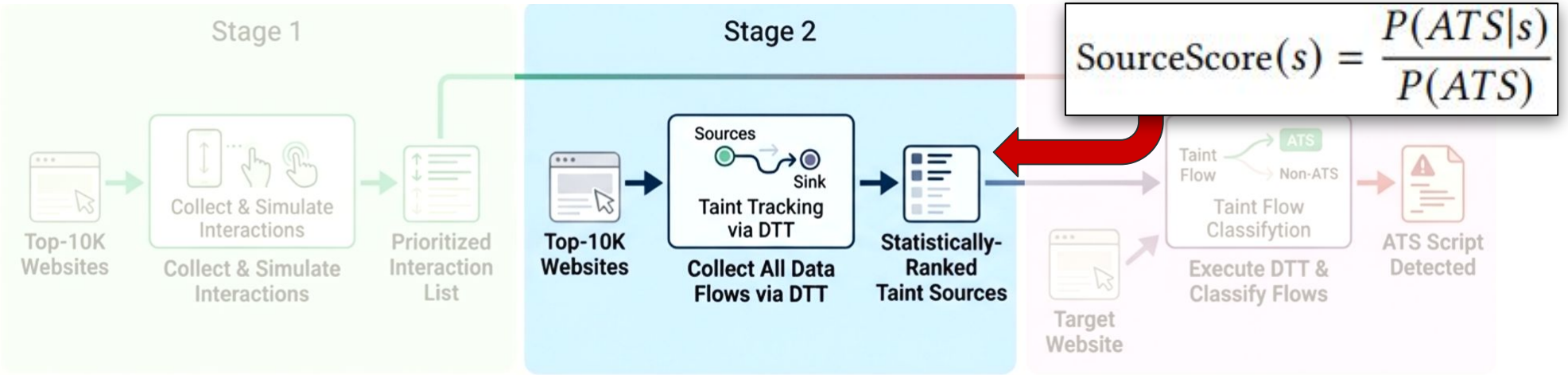
$$\text{JS Loading Ratio}(i) = \frac{\text{Occurrences of } i \text{ that trigger JS loads}}{\text{Total occurrences of } i}$$

Design: (2) Source Identifier

- **Challenge:** ATS scripts use a wide variety of APIs to collect information, making it difficult to manually define all taint sources of contamination
- **Solution:** Instead of relying on an empirical list, we automatically identify **highly ATS-relevant APIs** by calculating a statistically based **SourceScore**

⇒ We select **341 APIs** with a Lift SourceScore exceeding 1 (i.e., a strong positive correlation with ATS activity) as final taint sources

Stage 2. Source Identifier



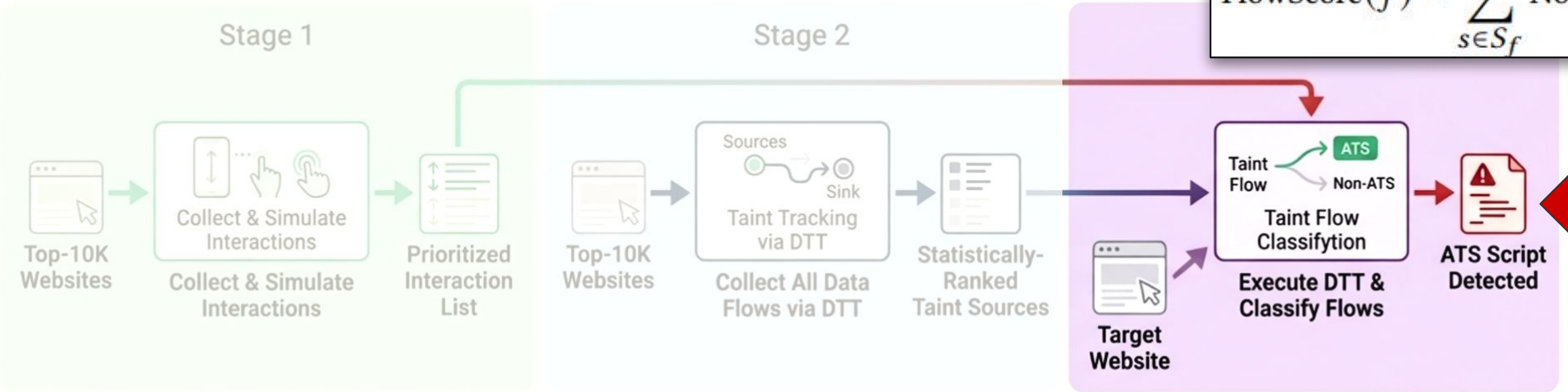
Design: (3) ATS Detector

- **Execute:** ADTT visits a target website, simulates interactions from the prioritized list. The engine monitors for data flows from the 341 identified Taint Sources to 26 known Taint Sinks (network APIs)
- **Classify:** For each detected flow, if the **FlowScore** exceeds a threshold, the flow is marked as an **ATS Flow**

⇒ Any JavaScript file involved in an ATS Flow is ultimately classified as an **ATS Script**)

Stage 3. ATS Detector

$$\text{FlowScore}(f) = \sum_{s \in S_f} \text{NormSourceScore}(s)$$



Evaluation Setup & Dataset

- **Implementation:**

Python 3.10

Black Widow
(Selenium-based Crawler)

PanoptiChrome
(Custom Chromium for DTT)

- **Evaluation Dataset:**

2,000

Randomly sampled 2K websites to avoid bias
- 1K from the top-10K
- 1K from the top 10K~100K

- **Comparison Targets:**

5 SOTA ML-based ATS blocking tools

(AdGraph, WebGraph, AdCPG, AdFlush, NoT.js)

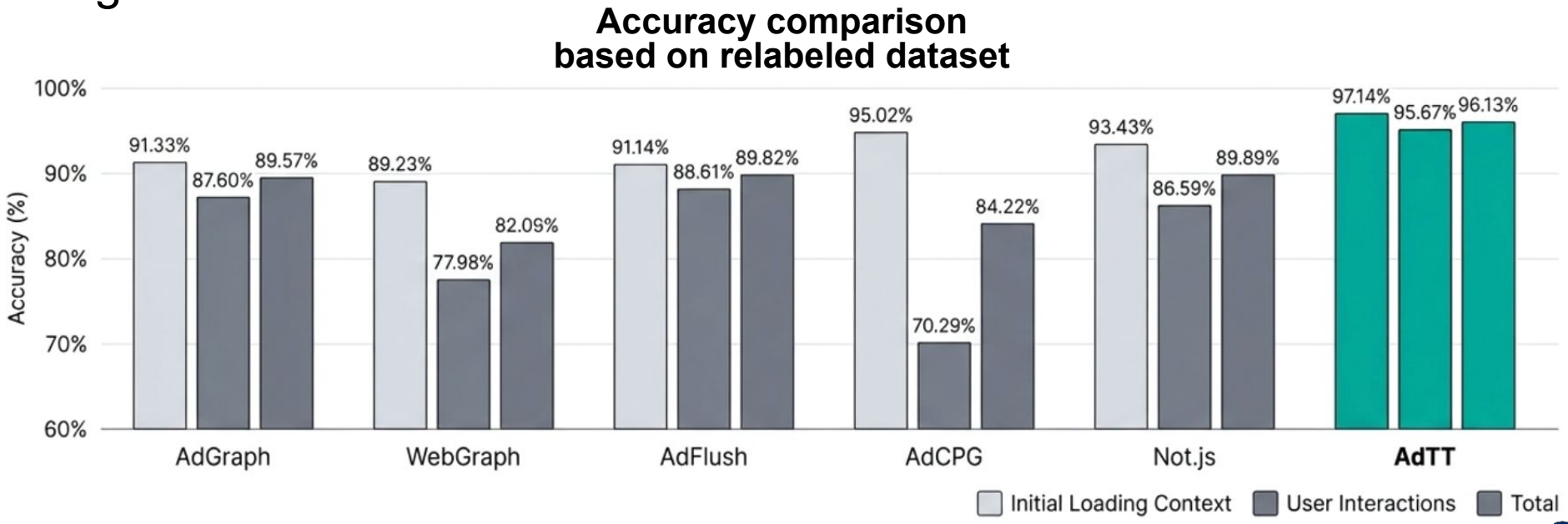
- **Ground Truth:**

EasyList and EasyPrivacy

To construct more robustness, we relabeled manually on FPs and FNs

Result: (1) Overall Detection Performance

- ADTT achieved **96.13% accuracy** on the reconstructed dataset, outperforming all state-of-the-art models
- While existing tools showed an average drop of 9.81% in interaction scenarios, ADTT maintained a high accuracy of **95.67%**
- As a result, ADTT detected **3.6 times more interaction-based ATS scripts** than existing tools



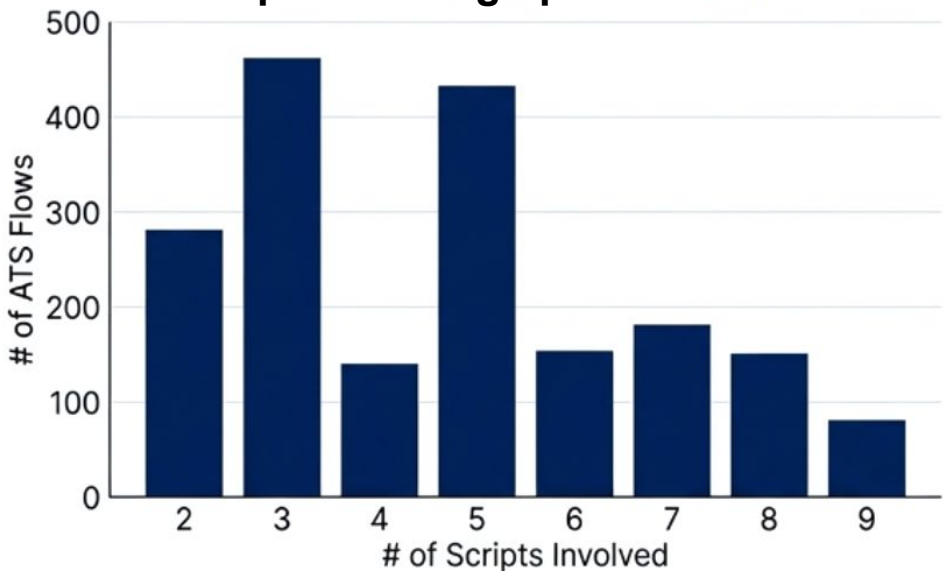
Result: (2) Analysis on Fragmented Script

- **Finding 1. Specialization:** Scripts are functionally specialized
- **Finding 2. Collaboration:** These specialized scripts frequently work together
- **Finding 3. Team Size:** A single ATS operation can involve between 2 and 9 different scripts working in concert

Collaboration between script categories

Source \ Sink	Communication	Navigation	Network
Environment	52	81	369
Behavior	6	16	84
Storage	15	1	24
DOM Access	11	11	317
Aggregation	6	9	53

Distribution of scripts per tracking operation flow



Result: (3) Case Study

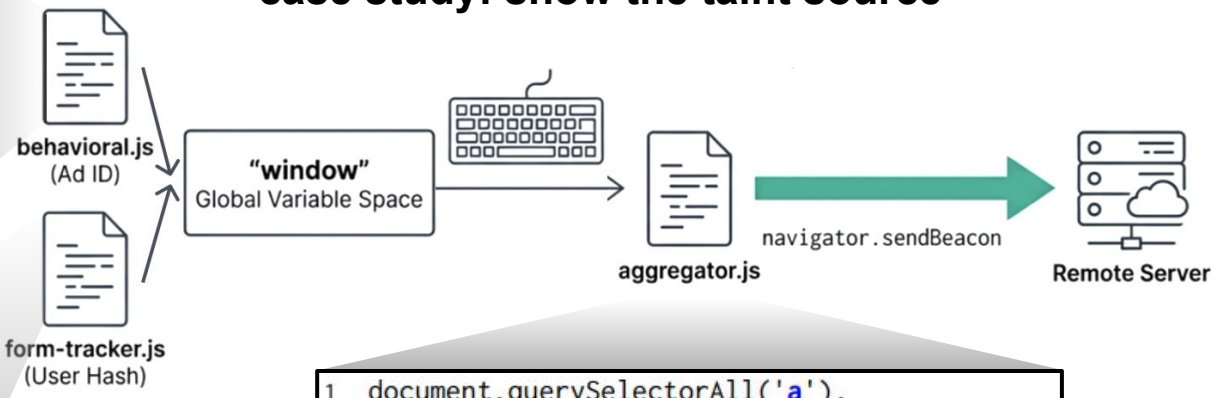
- **How ADTT Caught It:**

  ADTT's success came from simulating the *keyup* interaction and tracking the data from its source (*location.search*) to the *sendBeacon* sink

```
window.viewedAdIds = [];
const adMetaElement =
          document.querySelector('
             meta[name="ad-id"]');
const adId = adMetaElement.content;
setTimeout(() => {
   window.viewedAdIds.push(adId);
                          },4000);
```

```
window.userMD = null;
const urlParams =
      new URLSearchParams(
          window.location.search);
const userHash = urlParams.get('
                        user_hash');
window.userMD = userHash;
```

case study: show the taint source




```
document.querySelectorAll('a').
                        forEach(link => {
  const uMD = window.userMD;
  const ads = window.viewedAdIds.join(',');
  const newUrl = '${link.href}?
            user_md=${uMD}&viewed=${ads}';
  navigator.sendBeacon(newUrl, payload);});
```

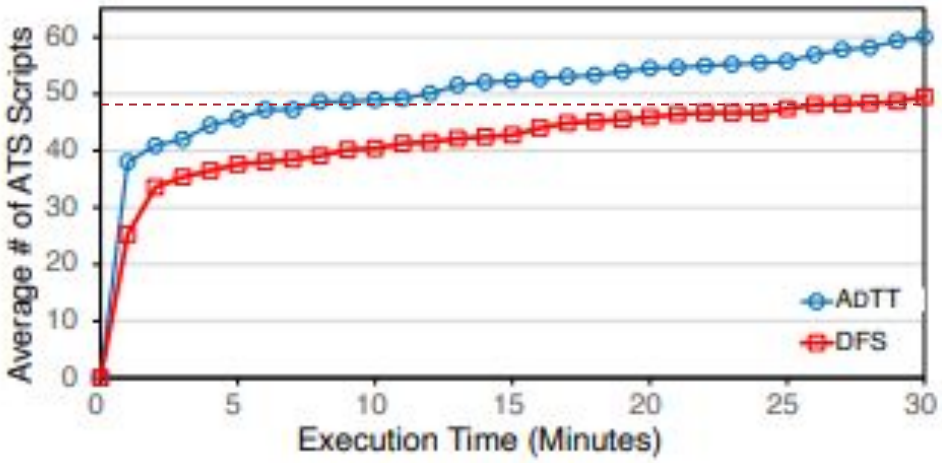
Result: (4) Ablation Study - Interaction

- **User interactions**
  - ➢ DFS-based **only detects 344 out of 452** unique ATS scripts
  - ➢ On the execution time manner, DFS requires 26 minutes to identify 48 ATS scripts, while ADTT required only 8 minutes
  - ➢ Within a 30-minute timeout, DFS detects only **82.5%** of ADTT

Results for ADTT's ablated variants

Approaches	Precision	Recall	Accuracy	# of ATS Scripts
ADTT	86.54%	**92.48%**	**96.13%**	452
Interaction: DFS	84.11%	85.00%	94.94%	344
Source: Subset-90%	89.25%	78.98%	94.61%	357
Source: Subset-80%	**89.47%**	71.46%	93.47%	323
Source: FP-Tracer	70.86%	27.43%	85.20%	124

The average number of ATS scripts detected by ADTT and DFS on a single page



Result: (4) Ablation Study - Taint Sources

- **Taint sources**
  - ➢ Subset-80% and -90% highly **degrades recall**
  - ➢ FP-Tracer, which is designed to detect browser fingerprinting, represents a narrow subset from ATS detections

Results for ADTT's ablated variants

Approaches	Precision	Recall	Accuracy	# of ATS Scripts
ADTT	86.54%	**92.48%**	**96.13%**	**452**
Interaction: DFS	84.11%	85.00%	94.94%	344
Source: Subset-90%	89.25%	78.98%	94.61%	357
Source: Subset-80%	**89.47%**	71.46%	93.47%	323
Source: FP-Tracer	70.86%	27.43%	85.20%	124

Limitations

- **Deployment:**
  - ➢ Dynamic taint tracking is **time-consuming**, so ADTT is designed for proactive unknown ATS discovery, not real-time blocking
  - ➢ ADTT runs on an instrumented **Chromium browser**. Detection scope is limited to APIs and interactions sets exposed by Chromium
- **Coverage:**
  - ➢ May miss ATS behaviors triggered by **sophisticated input** (e.g., specific URL parameters or complex from submissions)
  - ➢ Need integration of state-aware fuzzing or forced execution
- **Manual analysis:**
  - ➢ Required one-time **manual labeling** of JS files to derive the taint sources

Contributions

- **Exposed a Major Blind Spot:**
  First to systematically analyze and quantify the threat of interaction-triggered, fragmented ATS scripts
- **Novel Framework, ADTT:**
  The first tool to combine interaction simulation and DTT for robust ATS detection, achieving **96.13%** accuracy
- **Real-World Impact:**
  Identified **527** unknown ATS scripts across 11K websites, which we have reported to filter list maintainers
- **Reproducibility:**
  All artifacts will be released to support future research

Conclusion

The Problem

ATS are no longer static. They hide behind user interactions and use fragmented code to evade detection

The Limitation

State-of-the-art tools, focused on initial page load, have a critical blind spot for these dynamic behaviors, causing their accuracy to plummet

The Solution

AdTT integrates intelligent interaction simulation with dynamic taint tracking to reliably uncover hidden threats, outperforming existing methods and demonstrating its complementary role in ATS detection

Appendix A: Examples of ATS scripts

Execution Flow → Data Flow ---→

standalone_ats.js

```
1 const rumData = JSON.stringify({
2   id = Navigator.userAgent,
3   p: window.location.protocol,
4   s: {Screen.availLeft, availHeight}
5   t: performance.now(),
6   ...
7 });
8 navigator.sendBeacon(THIRD_PARTY_URL, rumData);
```

a.com/ats_A.js ❶

```
1 let observer = new PerformanceObserver((list)=>{
2   mkto = performanceEntries;
3 });
4 observer.getRecord('tracker', viewable);
5 document.addEventListener("onclick", (
6   t = document.createElement("script")
7   t.src = "https://b.com/ats_B.js"
8   document.body.appendChild(t)
9 ));
```

Click!

c.com/ats_C.js ❸

```
1 document.addEventListener("onscroll",(
2   i = document.createElement('Image")
3   i.src = "https://c.com/ats_D.js"
4   document.body.appendChild(t)
5 ));
```

Dynamically Loaded Content

Scroll!

b.com/atc_B.js ❷

```
1 const mid = hash(NavigatorUAData.userAgent &
2 NavigatorUAData.brands);
```

c.com/ats_D.js ❹

```
1 XMLHttpRequest.send(
2   "GET", ENDPOINT_URL + mid + mkto
3 );
```

Appendix B: 72 User Interaction List

- To prepare this list, we collected **124 event types** from HTML Standard
- We exclude 47 types since they do not depend on real user interaction
- We exclude 5 types that are not supported in Chrome or Selenium Action API

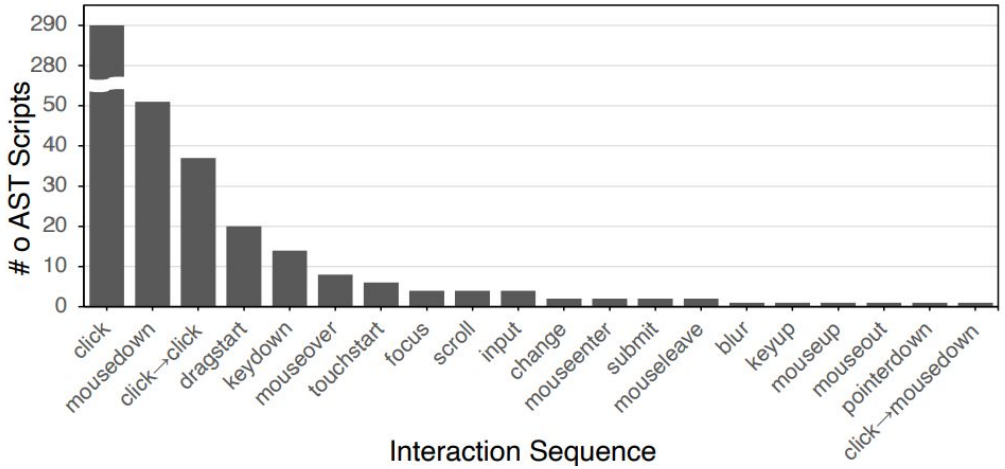
Category	Interaction Types
Mouse	click, dblclick, auxclick, mousedown, mouseup, mousemove, mouseover, mouseout, mouseenter, mouseleave, contextmenu, wheel, mousewheel, scroll, scrollend, longpress
Keyboard	keydown, keyup, keypress, compositionstart, compositionupdate, compositionend
Form	input, change, submit, reset, invalid, select, cut, copy, paste, search
Focus	focus, focusin, focusout, blur
Touch & Pointer	touchstart, touchmove, touchend, touchcancel, touch, tap, pointerdown, pointermove, pointerup, pointercancel, pointerover, pointerenter, pointerout, pointerleave, swipe
Drag & Drop	dragenter, drag, dragstart, dragend, dragover, dragleave, drop, toggle
Media	pause, seeking, seeked, timeupdate
Page	close, resize, fullscreenchange, hashchange, enterfullscreen
Volume	volume-button-clicked, volume-drag-released, volume-updated, volumechange

Appendix C: Prioritized Interaction List

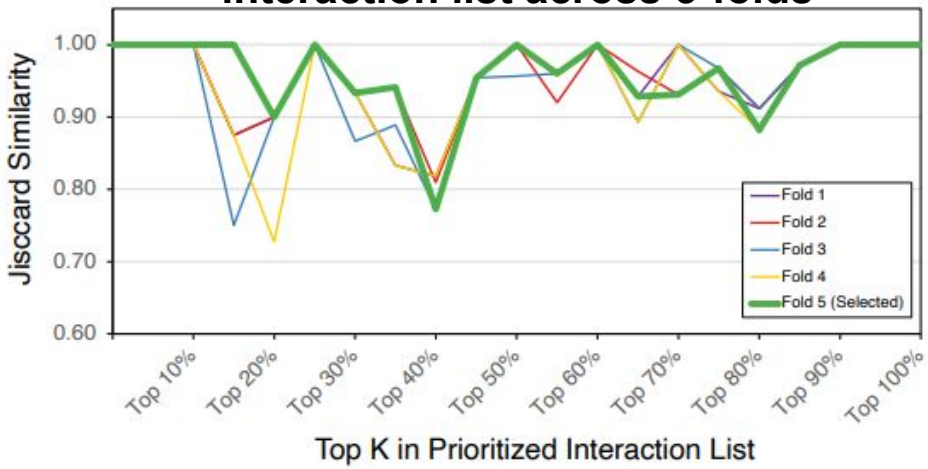
Prioritized interaction sequences, ordered by JS loading ratio (top 40)

Rank	Interaction Sequence	Total # of Occurrences	# of occurrences that trigger JS loads	JS Loading Ratio
1	mousedown	3,128	2,091	66.85
2	touchstart	576	310	53.82
3	scroll	354	188	53.11
4	click	49,247	25,042	50.85
5	input	690	344	49.86
6	dragstart	3,110	1,500	48.23
7	submit	357	163	45.66
8	mouseout	431	195	45.24
9	blur	472	211	44.70
10	keyup	475	212	44.63
11	mouseup	396	165	41.67
12	change	607	246	40.53
13	mouseleave	584	236	40.41
14	keydown	1,964	756	38.49
15	focus	1,204	458	38.04
16	pointerdown	323	119	36.84
17	click→mouseenter	154	54	35.06
18	mouseenter	857	299	34.89
19	change→click	178	62	34.83
20	click→mousedown	316	110	34.81
21	focusin	171	59	34.50
22	invalid	256	81	31.64
23	mouseover	3,107	930	29.93
24	mousemove	227	63	27.75
25	click→click	14,568	3,228	22.16
26	keydown	188	41	21.81
27	click→input	287	57	19.86
28	click→mouseover	648	127	19.60
29	click→keydown	680	118	17.35
30	click→change	325	56	17.23
31	click→mousemove	150	25	16.67
32	mouseover	283	46	16.25
33	mouseover→mouseover	284	45	15.85
34	click→submit	172	26	15.12
35	keydown→click	295	43	14.58
36	click→focus	266	38	14.29
37	click→dragstart	200	26	13.00
38	click→click→click	932	120	12.88
39	input→click	224	21	9.38
40	focus→click	185	16	8.65

Top 20 interaction sequences, ordered by the number of ATS script loads



Jaccard similarity of prioritized interaction list across 5-folds



Appendix D: Source and Sink Lists

List of 341 sources with their importance scores (currently, only the top 10 per category are shown)

Category	Sources (Source Score)
Environment	BaseAudioContext.state (1.131) / Blob.size (1.131) / Document.characterSet (1.131) / Document.charset (1.131) / DOMException.code (1.131) / DOMException.message (1.131) / DOMException.name (1.131) / DOMRect.x (1.131) / DOMRect.y (1.131) / DOMTokenList.length (1.131) / ...
Behavior	BaseAudioContext.currentTime (1.131) / Document.elementFromPoint (1.131) / Document.scrollingElement (1.131) / Document.all (1.131) / Document.activeElement (1.131) / ErrorEvent.filename (1.131) / Event.eventPhase (1.131) / Event.currentTarget (1.131) / Event.timeStamp (1.131) / EventSource.readyState (1.131) / ...
Storage	CookieChangeEvent.changed (1.131) / CookieChangeEvent.deleted (1.131) / IDBFactory.open (1.131) / IDBRequest.transaction (1.131) / IDBTransaction.error (1.131) / Window.sessionStorage (1.099) / Document.cookie (1.070) / Window.localStorage (1.070) / Storage.getItem (1.070) / CacheStorage.open (1.058) / ...
DOM Access	CSSStyleSheet.insertRule (1.131) / Document.documentURI (1.131) / Document.firstElementChild (1.131) / DocumentFragment.querySelectorAll (1.131) / Element.nextElementSibling (1.131) / Element.previousElementSibling (1.131) / Element.childElementCount (1.131) / HTMLElement.nonce (1.131) / Element.insertAdjacentElement (1.131) / Element.localName (1.131) / ...
Aggregation	Crypto.randomUUID (1.131) / ReadableStream.getReader (1.131) / TextEncoder.encode (1.120) / Location.toString (1.094) / Window.visualViewport (1.091) / Crypto.getRandomValues (1.090) / Window.crypto (1.090) / DOMParser.parseFromString (1.083) / Window.btoa (1.082) / TextDecoder.decode (1.050) / ...

List of 26 sinks

Category	Sinks
Communication	Client.postMessage, DedicatedWorkerGlobalScope.postMessage, MessagePort.postMessage, ServiceWorker.postMessage, Window.postMessage, Worker.postMessage
Navigation	History.pushState, History.replaceState, Window.open
Network	EventSource.url, HTMLElement.src, HTMLIframeElement.src, HTMLImageElement.src, HTMLLinkElement.href, HTMLObjectElement.data, XMLHttpRequest.send, HTMLScriptElement.src, Location.assign, Location.replace, Navigator.sendBeacon, RTCDataChannel.send, WebSocket.send,Window.fetch

Source important analysis (5-fold cross-validation)

Folds	Precision	Recall	Accuracy	# of ATS Scripts
Fold 1	94.52%	90.44%	91.68%	714
Fold 2	94.55%	93.34%	92.56%	730
Fold 3 (Selected)	96.51%	93.56%	**94.93%**	743
Fold 4	94.89%	90.16%	92.48%	725
Fold 5	96.04%	91.57%	92.88%	736

Appendix E: Detailed Performance

- ADTT achieved **a high accuracy rate of over 95% in both** initial loading and interaction contexts
- **67.13%** of ATS scripts loaded during user interactions were fragmented
- ADTT discovered **265 unknown ATS scripts** that were missed by existing filter lists

Comparison of ADTT against prior approaches

Tools	Initial Loading Context			User Interactions			Total		
	Precision	Recall	Accuracy	Precision	Recall	Accuracy	Precision	Recall	Accuracy
AdGraph	84.66%	69.30%	91.33%	40.54%	45.92%	87.60%	67.60%	61.98%	89.57%
WebGraph	82.02%	77.06%	89.23%	33.57%	39.41%	77.98%	54.86%	58.03%	82.09%
AdFlush	83.33%	72.87%	91.14%	40.59%	77.36%	88.61%	62.68%	74.22%	89.82%
AdCPG	77.29%	**86.49%**	95.02%	39.90%	26.19%	70.29%	59.25%	49.48%	84.22%
Not.js	84.47%	79.74%	93.43%	36.89%	61.79%	86.59%	61.41%	73.52%	89.89%
ADTT	**90.11%**	85.42%	**97.14%**	**85.71%**	**94.38%**	**95.67%**	**86.54%**	**92.48%**	**96.13%**

Overall results of ADTT on the 2K websites

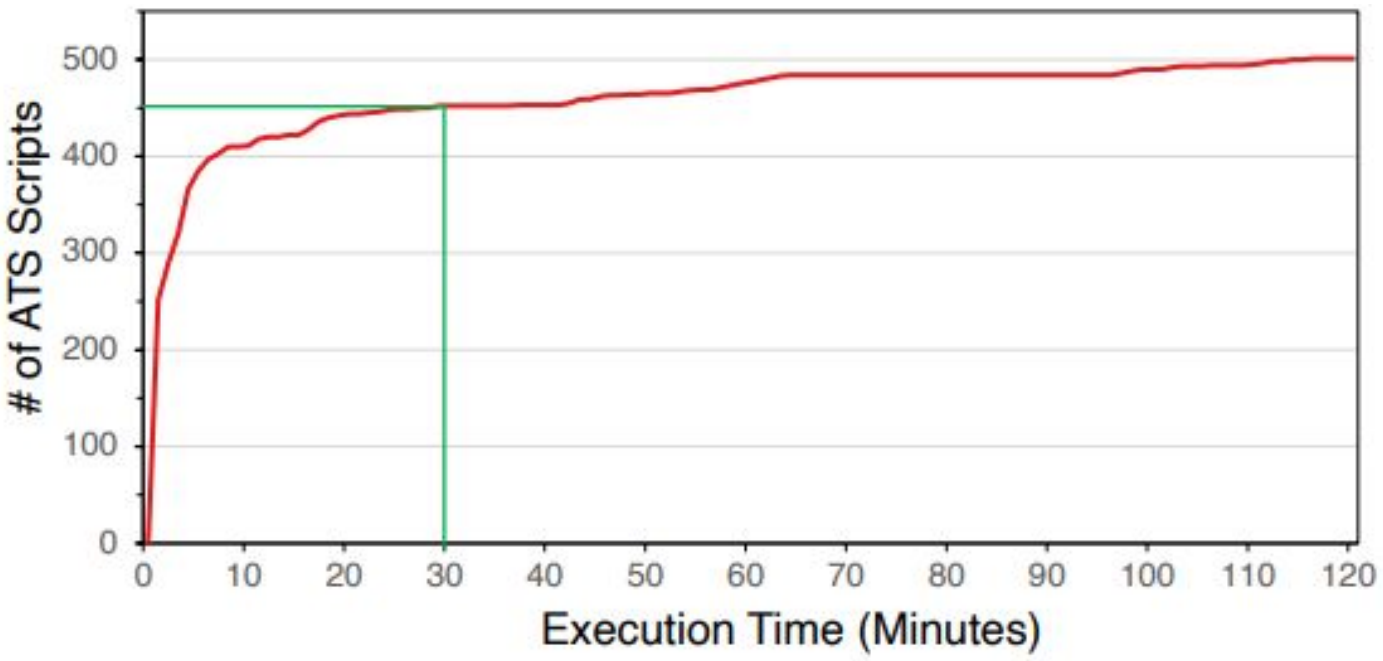
	Initial Context	User Interactions	Total
# of Valid Websites	-	-	1,851
Execution Time	-	-	6h 8m
# of Detected Flows	685	1,179	1,864
# of Loaded JS Files	804	1,756	2,560
Ground Truth Labelling with Filter Lists			
├ ATS Scripts	65	122	187
└ Non-ATS Scripts	739	1,634	2,373
Precision (Fixed Ver.)	58.24% (90.11%)	22.45% (85.71%)	29.19% (86.54%)
Recall (Fixed Ver.)	81.54% (85.41%)	72.13% (94.38%)	75.40% (92.48%)
Accuracy (Fixed Ver.)	93.78% (97.14%)	80.75% (95.67%)	84.84% (96.13%)
Ground Truth Relabelling Assisted by ADTT			
├ ATS Scripts	96	356	452
│ └ Fragmented Scripts	24 (25.00%)	239 (67.13%)	263 (58.19%)
└ Non-ATS Scripts	708	1,400	2,108

Appendix F: Categories of ATS Behaviors

Type	Category	Description
Sources: Collecting information	**Environment**	Extract user environment information (e.g., `userAgent`)
	Behavior	Record dynamic user activities (e.g., `ScrollX`)
	Storage	Read/write browser storage data (e.g., `cookie`)
	DOM Access	Read/write DOM element (e.g., `appendChild`)
	Aggregation	Aggregate user data (e.g., `btoa`)
Sinks: Sending information	**Communication**	Communicate between different contexts (e.g., `postMessage`)
	Navigation	Navigate webpages (e.g., `window.open`)
	Network	Send data using JS APIs (e.g., `fetch`)

Appendix G: CDF of Detection Time

- Evaluate the impact of execution time on ATS detection performance
- Approximately 90% of all ATS scripts were detected within the first 29 minutes
- 30 minutes timeout was set by this observation



Thank You for Your Time!